# MESCAL Spesifikasyonu

**Minimalistik Olarak Tasarlanmış Akıllı Sözleşme Birleştirme Dili**
*Sürüm 1.0.0 — Resmi Teknik Spesifikasyon*

---

## 1. Giriş

MESCAL (Minimalistically Envisioned Smart Contract Assembling Language), KristaTech (KRISTA) uyumlu blokzincirler için Akıllı Sözleşmeleri tanımlamak, birleştirmek ve serileştirmek üzere tasarlanmış JSON tabanlı bildirimsel (declarative) bir dildir.

Geleneksel akıllı sözleşme dilleri (Solidity veya Plutus gibi) uzman yazılımcılar, ağır çalışma zamanı sanal makineleri (VM'ler) gerektirir ve önemli güvenlik açığı yüzeyleri oluşturur. MESCAL, **KISS (Keep It Simple, Stupid - Yalın Tut, Aptalca Olsun)** prensibine bağlı kalarak bu sorunları çözer: Bitcoin benzeri yığın betiği (stack script) talimatlarının ( `CScript` ) üst düzey, insan tarafından okunabilir ve makine tarafından ayrıştırılabilir bir JSON temsilidir.

### Neden MESCAL?

1. **Erişilebilirlik**: Kullanıcıların ve uygulamaların, düşük seviyeli assembly veya derleme bilgisine ihtiyaç duymadan akıllı sözleşmeler oluşturmasına, imzalamasına ve yürütmesine olanak tanır.
2. **Güvenlik**: Blokzincir opcode'ları ile eşleşen kısıtlı, belirleyici (deterministic) bir talimat kümesi üzerinde çalışarak yürütme zamanı re-entrancy (yeniden giriş) saldırılarını ve gas hesaplama sorunlarını ortadan kaldırır.
3. **Birlikte Çalışabilirlik**: JSON, neredeyse tüm modern programlama dilleri tarafından yerel olarak ayrıştırılabilir ve üretilebilir; bu da tarayıcı (explorer) entegrasyonunu, istemci tarafı cüzdanları ve arka uç servislerini oldukça basit hale getirir.

---

## 2. Temel Felsefe ve JSON Biçimlendirmesi

Her MESCAL betiği, bildirimsel bir JSON ağacıdır.

- **Sıralama Hassasiyeti**: Diziler ve JSON nesneleri, göründükleri sıraya göre (ilk elemandan son elemana doğru) kesin bir şekilde ayrıştırılır.
- **Kendi Kendine Yeten Şemalar**: Bir sözleşme tanımı, doğrudan doğrusal blokzincir bayt koduna (bytecode) derlenir.
- **Üst Düzey Değişkenler**: JSON anahtarları parametre bağlamaları olarak işlev görür ve zincir dışı (off-chain) istemcilerin kullanıcı girdilerini (anahtarlar, zaman damgaları veya sınırlar gibi) işlem gönderilmeden önce doğrudan şablonlara bağlamasına olanak tanır.

---

## 3. Yapısal Spesifikasyon ve Bileşen Türleri

Bir MESCAL programı üç temel yapısal türden oluşur: `basic` , `condition` ve `contract` .

### 3.1. Temel Öğeler ( `basic` )

Bir `basic` bloğu, ilkel bir betik operatörünü veya statik değer sarmalayıcısını temsil eder. Doğrudan bir veya daha fazla Bitcoin betik opcode'u ile eşleşir.

```
{
  "type": "basic",
  "name": "Unique-Identifier",
  "role": "<op-code-role>",
  "inputs": [
    {
      "type": "<data-type>",
      "name": "Parameter-Name",
      "value": "<data-value>"
    }
  ]
}
```

#### Desteklenen Rol ve Opcode'lar

- `hash160` : Önce SHA256, ardından RIPEMD160 uygular. Varsayılan olarak `<val> OP_HASH160` ile, `"mode": "htlc-equal"` (veya girdi `"name"` / `"type"` alanı `"hash160"` ve değeri 20 baytlık hex) olduğunda ise `OP_HASH160 <20-byte-hash> OP_EQUAL` ile eşleşir.
- `check-signature-verification` : Bir imzayı açık anahtara (public key) karşı doğrular. Varsayılan olarak `<pubkey> OP_CHECKSIGVERIFY` ( `"verify": true` ), `"verify": false` belirtildiğinde ise `<pubkey> OP_CHECKSIG` ile eşleşir.
- `equalverify-checksig` : Açık anahtar hash'inin (public key hash) girdiyle eşleştiğini doğrular, ardından imzayı doğrular. `OP_DUP OP_HASH160 <pubkeyhash> OP_EQUALVERIFY OP_CHECKSIG` ile eşleşir.
- `number` : Yığına sayısal bir sabit yerleştirir. Standart betik tam sayıları (örneğin `OP_1` , `OP_2` veya bayt dizileri) ile eşleşir.
- `multi-signature` : Açık anahtarları yerleştirir ve çoklu imza (multisig) doğrulamasını zorunlu kılar. `<m> <pubkeys...> <n> OP_CHECKMULTISIG` ile eşleşir.
- `lock-time` : İşlem harcamalarını belirli bir hedef zamana veya blok yüksekliğine kadar kilitler. `<locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP` ile eşleşir.

---

### 3.2. Koşul Öğeleri ( `condition` )

Bir `condition` bloğu, koşullu yürütme dallarını (karşılaştırmalar ve mantıksal değerlendirmeler) kullanarak kontrol akışını uygular.

```
{
  "type": "condition",
  "name": "Unique-Condition-Name",
  "role": "if-condition",
  "expressions": [
    {
      "required": true,
      "type": "basic",
      "name": "Trigger-Variable"
    }
  ],
  "true": [
    { "type": "basic", "name": "Success-Branch-Action" }
  ],
  "false": [
    { "type": "basic", "name": "Failure-Branch-Action" }
  ]
}
```

- **Betik Eşleme**: Arka planda bu yapı, `OP_IF ... OP_ELSE ... OP_ENDIF` yapılarına derlenir.

---

### 3.3. Birleştirilmiş Sözleşmeler ( `contract` )

Bir `contract`, sırayla değerlendirilen eylemleri ( `basic` veya `condition` blokları) içeren en üst düzey şemadır. İşlem çıktılarını (UTXO'lar) kilitlemek için kullanılan nihai betik şablonu olarak işlev görür.

```
{
  "type": "contract",
  "name": "My-Contract-Name",
  "description": "Human readable documentation for this contract.",
  "actions": [
    { "type": "basic", "name": "Step-1-Action" },
    { "type": "condition", "name": "Step-2-Conditional" }
  ]
}
```

---

## 4. Eksiksiz Spesifikasyonlar ve Örnekler

### 4.1. HASH160 Hesaplaması

- **CScript Karşılığı**: `<inputs[0]> OP_HASH160`

```
{
  "type": "basic",
  "role": "hash160",
  "name": "Hash-Validation",
  "inputs": [
    {
      "type": "string-or-number",
      "name": "Data-to-Hash",
      "value": "KristaTechSmartContractInput"
    }
  ]
}
```

> **NOT**
>
> `src/script/mescal.cpp` içerisindeki `CompileBasicBlock` tarafından desteklenen ek yerleşik `basic` rolleri şunlardır:
>
> - `check-signature-verification`: `<pubkey> OP_CHECKSIGVERIFY` (veya `"verify": false` olduğunda `<pubkey> OP_CHECKSIG`)
> - `number`: `<int64_t>`
> - `add`, `sub`, `bool-and`, `bool-or`, `num-equal`, `greater-than`, `drop`: `OP_ADD`, `OP_SUB`, `OP_BOOLAND`, `OP_BOOLOR`, `OP_NUMEQUAL`, `OP_GREATERTHAN`, `OP_DROP`
> - `coin-lock-miner`: `<pubkey> OP_DROP <locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_DUP OP_HASH160 <pubkeyhash> OP_EQUALVERIFY OP_CHECKSIG`
> - `pow-miner`: `<nonce> <challenge> <pubkey> OP_DROP OP_DROP OP_DROP <locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_DUP OP_HASH160 <pubkeyhash> OP_EQUALVERIFY OP_CHECKSIG`

---

### 4.2. Pay-to-Public-Key-Hash (P2PKH) karşılığı

- **CScript Karşılığı**: `OP_DUP OP_HASH160 <pubkeyhash> OP_EQUALVERIFY OP_CHECKSIG`

```
{
  "type": "basic",
  "role": "equalverify-checksig",
  "name": "Standard-P2PKH",
  "inputs": [
    {
      "type": "pubkeyhash",
      "name": "Recipient-PubkeyHash",
      "value": "b5a9c9f285d893ce71ab9de8f5c09d765ee982ba"
    }
  ]
}
```

---

### 4.3. Çoklu İmza (2-of-2)

- **CScript Karşılığı**: `2 <pubkey1> <pubkey2> 2 OP_CHECKMULTISIG`

```
{
  "type": "basic",
  "role": "multi-signature",
  "name": "Escrow-Multisig",
  "inputs": [
    {
      "type": "number",
      "name": "n",
      "value": 2
    },
    {
      "type": "number",
      "name": "m",
      "value": 2
    },
    {
      "type": "array",
      "name": "Signatures",
      "value": [
        "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f",
        "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a"
      ]
    }
  ]
}
```

---

### 4.4. Zaman Kilitli Mevduat (CLTV)

- **CScript Karşılığı**: `<expiry-time> OP_CHECKLOCKTIMEVERIFY OP_DROP`

```
{
  "type": "basic",
  "role": "lock-time",
  "name": "Time-Lock-Enforcement",
  "inputs": [
    {
      "type": "timestamp-or-block-height",
      "name": "Lock-Until",
      "value": 1780718400
    }
  ]
}
```

---

### 4.5. Koşullu Yürütme (If-Else)

- **CScript Karşılığı**: `OP_2 OP_IF <expiry-time> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_1 OP_ELSE <Data-to-Hash> OP_HASH160 OP_ENDIF`

```
{
  "type": "condition",
  "role": "if-condition",
  "name": "Multi-Path-Resolution",
  "expressions": [
    {
      "required": true,
      "type": "basic",
      "name": "Constant-Value-2"
    }
  ],
  "true": [
    {
      "type": "basic",
      "name": "Time-Lock-Enforcement"
    },
    {
      "type": "basic",
      "name": "Constant-Value-1"
    }
  ],
  "false": [
    {
      "type": "basic",
      "name": "Hash-Validation"
    }
  ]
}
```

---

### 4.6. Birleştirilmiş Fon Dondurma Sözleşmesi

Hedef tarihe kadar fonları bir UTXO içinde kilitler. Hedef tarih geçtikten sonra, alıcı kendi imzasını sağlayarak bu fonları harcayabilir.

- **CScript Karşılığı**: `<expiry-time> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG`

```
{
  "type": "contract",
  "name": "Freezing-Funds",
  "description": "Freezes coins in a UTXO until a future timestamp, then allows spending with
      recipient key.",
  "actions": [
    {
      "type": "basic",
      "name": "Time-Lock-Enforcement"
    },
    {
      "type": "basic",
      "name": "Standard-P2PKH"
    }
  ]
}
```

---

### 4.7. Dead Man’s Switch (Miras)

Sahibin gizli anahtarı fonları hareket ettirmek için kullanılmazsa, belirli bir blok yüksekliğinden veya zaman damgasından (süre bitiminden) sonra, varis kendi imzasıyla fonları talep edebilir.

- **CScript Karşılığı**: `<heir-pubkey> OP_CHECKSIG OP_IF <expiry> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_1 OP_ELSE <owner-pubkey> OP_CHECKSIG OP_ENDIF`

```
{
  "basic": {
    "Heir-Sig": {
      "role": "check-signature-verification",
      "verify": false,
      "inputs": [
        {
          "name": "Pubkey",
          "type": "pubkey",
          "value": "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f"
        }
      ]
    },
    "Lock-Time": {
      "role": "lock-time",
      "inputs": [
        {
          "name": "Lock-Until",
          "type": "timestamp-or-block-height",
          "value": 1780718400
        }
      ]
    },
    "Push-True": {
      "role": "number",
      "inputs": [
        {
          "name": "True",
          "type": "number",
          "value": 1
        }
      ]
    },
    "Owner-Sig": {
      "role": "check-signature-verification",
      "verify": false,
      "inputs": [
        {
          "name": "Pubkey",
          "type": "pubkey",
          "value": "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a"
        }
      ]
    }
  },
  "condition": {
    "Inheritance-Condition": {
      "role": "if-condition",
      "expressions": [
        { "type": "basic", "name": "Heir-Sig" }
      ],
      "true": [
        { "type": "basic", "name": "Lock-Time" },
        { "type": "basic", "name": "Push-True" }
      ],
      "false": [
        { "type": "basic", "name": "Owner-Sig" }
      ]
    }
  },
  "contract": {
    "Inheritance-Switch": {
```

```
      "description": "Allows heir to spend after lock time passes, otherwise owner can spend
        anytime.",
      "actions": [
        { "type": "condition", "name": "Inheritance-Condition" }
      ]
    }
  },
  "active_contract": "Inheritance-Switch"
}
```

---

### 4.8. Arabuluculu Çift İmzalı Escrow (2-of-3)

Alıcı, Satıcı ve Arabulucunun anahtarları elinde tuttuğu standart bir escrow sözleşmesi. Kilitli fonları serbest bırakmak veya iade etmek için 3 taraftan herhangi 2'si imzalayabilir.

- **CScript Karşılığı**: `2 <buyer-pubkey> <seller-pubkey> <mediator-pubkey> 3 OP_CHECKMULTISIG`

```
{
  "basic": {
    "Escrow-2of3": {
      "role": "multi-signature",
      "inputs": [
        { "name": "m", "type": "number", "value": 2 },
        { "name": "n", "type": "number", "value": 3 },
        {
          "name": "Signatures",
          "type": "array",
          "value": [
            "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f",
            "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a",
            "02cd98ef1234a567bcde0123ef5678cd12345678ab12345678cd12345678ef1234"
          ]
        }
      ]
    }
  },
  "contract": {
    "Escrow-2of3-Contract": {
      "description": "2-of-3 Escrow Contract between Buyer, Seller, and Mediator.",
      "actions": [
        { "type": "basic", "name": "Escrow-2of3" }
      ]
    }
  },
  "active_contract": "Escrow-2of3-Contract"
}
```

---

### 4.9. 2 Faktörlü Kimlik Doğrulama (2FA) Güvenlik Cüzdanı

Hem kullanıcının birincil mobil cüzdanından hem de ikincil donanım cüzdanından imza gerektiren operasyonel bir güvenlik politikası.

- **CScript Karşılığı**: `2 <phone-pubkey> <hardware-pubkey> 2 OP_CHECKMULTISIG`

```
{
  "basic": {
    "Multisig-2of2": {
      "role": "multi-signature",
      "inputs": [
        { "name": "m", "type": "number", "value": 2 },
        { "name": "n", "type": "number", "value": 2 },
        {
          "name": "Signatures",
          "type": "array",
          "value": [
            "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f",
            "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a"
          ]
        }
      ]
    }
  },
  "contract": {
    "Security-Wallet-2FA": {
      "description": "Requires signatures from both primary wallet (mobile) and secondary
        backup (hardware wallet).",
      "actions": [
        { "type": "basic", "name": "Multisig-2of2" }
      ]
    }
  },
  "active_contract": "Security-Wallet-2FA"
}
```

---

### 4.10. Hash Time-Locked Swap (HTLC)

Klasik bir atomik zincirler arası takas (atomic cross-chain swap) sözleşmesi. Alıcı, hash değeri `H` olan gizli preimage'ı sağlayarak fonları anında talep edebilir. Eğer önce zaman aşımı süresi dolarsa, gönderici tam para iadesi alabilir.

- **CScript Karşılığı**: `OP_HASH160 <hash> OP_EQUAL OP_IF <recipient-pubkey> OP_CHECKSIG OP_ELSE <expiry> OP_CHECKLOCKTIMEVERIFY OP_DROP <sender-pubkey> OP_CHECKSIG OP_ENDIF`

```
{
  "basic": {
    "Preimage-Check": {
      "role": "hash160",
      "mode": "htlc-equal",
      "inputs": [
        {
          "name": "Hash160",
          "type": "string-or-number",
          "value": "b5a9c9f285d893ce71ab9de8f5c09d765ee982ba"
        }
      ]
    },
    "Recipient-Sig": {
      "role": "check-signature-verification",
      "verify": false,
      "inputs": [
        {
          "name": "Pubkey",
          "type": "pubkey",
          "value": "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f"
        }
      ]
    },
    "Timeout-Check": {
      "role": "lock-time",
      "inputs": [
        {
          "name": "Lock-Until",
          "type": "timestamp-or-block-height",
          "value": 1780718400
        }
      ]
    },
    "Sender-Sig": {
      "role": "check-signature-verification",
      "verify": false,
      "inputs": [
        {
          "name": "Pubkey",
          "type": "pubkey",
          "value": "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a"
        }
      ]
    }
  },
  "condition": {
    "HTLC-Branch": {
      "role": "if-condition",
      "expressions": [
        { "type": "basic", "name": "Preimage-Check" }
      ],
      "true": [
        { "type": "basic", "name": "Recipient-Sig" }
      ],
      "false": [
        { "type": "basic", "name": "Timeout-Check" },
        { "type": "basic", "name": "Sender-Sig" }
      ]
    }
  },
  "contract": {
```

```
    "Atomic-Swap-HTLC": {
      "description": "Atomic Swap HTLC: Claimable immediately with secret preimage, or
        refundable to sender after timeout.",
      "actions": [
        { "type": "condition", "name": "HTLC-Branch" }
      ]
    }
  },
  "active_contract": "Atomic-Swap-HTLC"
}
```

---

### 4.11. Çok Yollu Güvenlik Kurtarması

Sahibin imzası fonları her an harcayabilir. Sahibin anahtarı kaybolursa, yedek kurtarma ekibi (güvenilir arkadaşların/hizmetlerin 2-of-3 multisig'i) fonları kurtarabilir, ancak sahibinin olası kötü niyetli kurtarma girişimlerini engellemesine izin vermek için yalnızca 30 günlük bir gecikmeden sonra.

- **CScript Karşılığı**: `2 <friend1> <friend2> <backup> 3 OP_CHECKMULTISIG OP_IF <recovery-delay> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_1 OP_ELSE <owner-pubkey> OP_CHECKSIG OP_ENDIF`

```
{
  "basic": {
    "Owner-Sig": {
      "role": "check-signature-verification",
      "verify": false,
      "inputs": [
        {
          "name": "Pubkey",
          "type": "pubkey",
          "value": "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f"
        }
      ]
    },
    "Recovery-Delay": {
      "role": "lock-time",
      "inputs": [
        {
          "name": "Lock-Until",
          "type": "timestamp-or-block-height",
          "value": 1780718400
        }
      ]
    },
    "Push-True": {
      "role": "number",
      "inputs": [
        {
          "name": "True",
          "type": "number",
          "value": 1
        }
      ]
    },
    "Recovery-Multisig": {
      "role": "multi-signature",
      "inputs": [
        { "name": "m", "type": "number", "value": 2 },
        { "name": "n", "type": "number", "value": 3 },
        {
          "name": "Signatures",
          "type": "array",
          "value": [
            "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a",
            "02cd98ef1234a567bcde0123ef5678cd12345678ab12345678cd12345678ef1234",
            "03ab89ef1234a567bcde0123ef5678cd12345678ab12345678cd12345678ef1235"
          ]
        }
      ]
    }
  },
  "condition": {
    "Recovery-Path": {
      "role": "if-condition",
      "expressions": [
        { "type": "basic", "name": "Recovery-Multisig" }
      ],
      "true": [
        { "type": "basic", "name": "Recovery-Delay" },
        { "type": "basic", "name": "Push-True" }
      ],
      "false": [
        { "type": "basic", "name": "Owner-Sig" }
```

```
      ]
    }
  },
  "contract": {
    "Multi-Path-Recovery": {
      "description": "Owner can spend anytime. Recovery team (2-of-3 multisig) can recover
        funds only after a recovery lock delay.",
      "actions": [
        { "type": "condition", "name": "Recovery-Path" }
      ]
    }
  },
  "active_contract": "Multi-Path-Recovery"
}
```

---

## 5. Derleyici Uygulama Yönergeleri

MESCAL sözleşmelerini KristaTech (KRISTA) düğümlerine (nodes) dağıtmak için, bir ayrıştırıcının (parser) JSON ağacını işlemesi ve serileştirilebilir `CScript` baytlarını üretmesi gerekir.

### 5.1. Ayrıştırma Sırası

Derleyici, en üst düzeydeki `"actions"` dizisini sırayla ayrıştırmalıdır:

1. Eylem tanımlarını özyinelemeli (recursively) olarak yükleyin.
2. Başvurulan parametreleri çözün ve değişkenleri enjekte edin.
3. Eşleşen opcode karşılıklarını bayt derleme akışına (byte compilation stream) aktarın.

### 5.2. Doğrulama Kuralları

CScript bayt kodunu (bytecode) üretmeden önce, derleyici aşağıdaki güvenlik kontrollerini doğrulamalıdır:

- **Yığın Derinliği Sınırı (Stack Depth Limit)**: Üretilen opcode'ların blokzincirin maksimum yığın derinliği sınırını (genellikle 1000 öge) aşmadığını doğrulayın.
- **Devre Dışı Bırakılmış Opcode'lar**: Bellek tükenmesini (memory exhaustion) önlemek için mutabakat katmanında (consensus layer) devre dışı bırakılmış yasaklı opcode'ların (örneğin `OP_CAT` veya `OP_LSHIFT`) enjekte edilmesini önleyin.
- **Zaman Kilidi Kontrolleri**: İşlem bükülebilirliği (transaction malleability) sorunlarını önlemek için, herhangi bir `lock-time` eyleminin, zaman kilidi opcode'unu (`OP_CHECKLOCKTIMEVERIFY`) imza kontrolünden önce yerleştirdiğinden emin olun.

---

## 6. Taproot (P2TR) Script-Path Entegrasyonu

MESCAL sözleşmeleri, doğrudan Taproot (P2TR) script-path harcama koşullarına derlenebilir. Bu, geliştiricilerin sözleşme betiklerini bir Taproot çıktısı içinde gizlemesine ve bunları yalnızca bir script-path harcaması sırasında açığa çıkarmasına olanak tanır.

### 6.1. `compilemescaltotaproot` RPC Komutu

Daemon, bir MESCAL sözleşmesini derlemek ve gerekli Taproot parametrelerini üretmek için yerleşik bir `compilemescaltotaproot` RPC komutu sağlar.

#### Komut Bağımsız Değişkenleri

1. `json` (string, gerekli): MESCAL sözleşmesi JSON dizesi.
2. `internal_pubkey` (string, gerekli): Key-path harcaması ana anahtarını (master key) temsil eden, 32 baytlık (64 karakter) hex kodlu bir x-only açık anahtar.

#### Komut Örneği

```
kristatech-cli compilemescaltotaproot '{"basic": {"MyDrop": {"role": "drop"}}, "contract":
        {"TestDrop": {"actions": [{"type": "basic", "name": "MyDrop"}]}}, "active_contract":
        "TestDrop"}' "697caf5a1ea29fa2e41e32ce0514e6c11d0b549be82136369ce8d1d435e9e731"
```

#### Komut Sonuç Nesnesi

- `address` (string): Bech32m kodlu Taproot (P2TR) adresi.
- `scriptPubKey` (string): İşlem çıktısı için kilitleme scriptPubKey değeri.
- `leafScript` (string): Derlenmiş MESCAL betiğinin hex değeri.
- `controlBlock` (string): Leaf script'in Taproot taahhüdüne (commitment) dahil edildiğini kanıtlayan witness kontrol bloğu (control block) (33 bayt hex).

---

### 6.2. P2TR Script-Path UTXO'larını Harcama

Bir Taproot adresinde kilitli olan bir UTXO'yu script-path aracılığıyla (derlenmiş MESCAL sözleşmesini kullanarak) harcamak için:

1. **Ham harcama işlemini (raw spending transaction) oluşturun**: Girdiler (inputs), fonlanmış P2TR çıktısını harcamalıdır.

2. **scriptSig witness parametrelerini enjekte edin**: Harcama girdisinin `scriptSig` alanında şu bayt dizisini oluşturun: `[witness_stack_items...] + [leafScript] + [controlBlock]`

   *Not: KristaTech VM yorumlayıcısında, bir Taproot script-path harcaması için scriptSig, önce sözleşme yürütme argümanlarını alır, ardından leaf script itilmesini (push) ve ardından kontrol bloğunun (control block) itilmesini takip eder.* `SIGVERSION_TAPSCRIPT` *yaprak betikleri içerisinde* `OP_CHECKSIG` */* `OP_CHECKSIGVERIFY` */* `OP_CHECKMULTISIG` *hem 32 baytlık x-only BIP340 Schnorr açık anahtarlarını hem de standart 33/65 baytlık MESCAL ECDSA açık anahtarlarını (*`SIGVERSION_BASE` *DER ECDSA doğrulamasına otomatik yönlendirilerek) destekler.*

### scriptSig Harcama Yapısı Örneği (Python)

```
# witness_stack_items: MyDrop (OP_DROP) üstteki elemanı attığında yığında OP_1 (true) kalması
        için iki adet OP_1 (0x5151)
witness_stack = bytes.fromhex("5151")
# leafScript: Bölüm 6.1'deki MyDrop eyleminden derlenen OP_DROP (0x75)
leaf_script = bytes.fromhex("75")
# controlBlock: RPC tarafından döndürülen 33 baytlık kontrol bloğu
control_block = bytes.fromhex(control_block_hex)

# Nihai scriptSig oluşturma:
# Witness parametrelerini, leaf script'i ve kontrol bloğunu yığına iter
script_sig = witness_stack + bytes([len(leaf_script)]) + leaf_script +
        bytes([len(control_block)]) + control_block
```